MODULE 5

Claude Code for Automation & Scripting

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Fifth**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Use Claude Code's headless mode (claude -p) to run AI-assisted tasks non-interactively from scripts and pipelines
2. Choose the appropriate output format (text, JSON, stream-json) for different automation scenarios and parse results programmatically
3. Pipe data into Claude Code and chain it with Unix utilities for data processing workflows
4. Apply --append-system-prompt and --allowedTools flags to enforce team standards and security constraints in automated runs
5. Build practical automation scripts for TI use cases: batch CSV processing, log analysis, code generation, and report creation
6. Design multi-turn automated sessions using --continue and --resume for complex, multi-step workflows

SESSION TIMING BREAKDOWN

0:00 – 0:10	**Section 1: Headless Mode Fundamentals — The -p Flag**
0:10 – 0:18	**Section 2: Output Formats — Text, JSON & Streaming**
0:18 – 0:28	**Section 3: Piping, Chaining & Unix Integration**
0:28 – 0:36	**Section 4: System Prompts, Permissions & Safety**
0:36 – 0:43	**Section 5: Real-World Automation Recipes for TI**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Lab 1**

0:00 – 0:10	SECTION 1: Headless Mode Fundamentals — The -p Flag

1. Headless Mode Fundamentals: The -p Flag

1.1 What Is Headless Mode?

In Module 4, you learned to use Claude Code interactively — typing prompts in a conversational session. Headless mode flips this model: instead of a live conversation, you give Claude a single instruction from a script, it executes the task, and returns the result to stdout. No interactive UI, no permission prompts, no back-and-forth.

This is the bridge between “developer using Claude Code at a desk” and “Claude Code running as part of an automated system.” For process engineers, headless mode turns Claude Code into a batch processing tool for CSV files, log analysis, and data validation. For IT engineers, it’s how you integrate AI into CI/CD pipelines, pre-commit hooks, and deployment scripts.

Interactive Mode	Headless Mode (-p)
You type prompts live in a session	Script passes a prompt as a command argument
Claude asks permission before actions	Permissions pre-configured via flags
Conversation persists until you exit	Single execution: run, return result, exit
Rich terminal UI with diffs and status	Plain text or JSON output to stdout
Best for: development, exploration, learning	Best for: automation, CI/CD, batch processing
Uses context window across prompts	Each invocation is independent (unless --continue)

1.2 Basic Syntax

The fundamental command is simple — add -p (short for --print) to any claude command:

```
# Basic headless execution
claude -p "Explain the main function in src/app.py"

# The -p flag is short for --print:
claude --print "Count the TODO comments in this project"

# Combine with other CLI flags:
claude -p "Analyze this code for bugs" --model opus --verbose

# Pipe from stdin:
echo "Check this function for errors" | claude -p

# Save output to file:
claude -p "Generate a README for this project" > README.md
```

☆ **KEY POINT:** The -p flag transforms Claude Code from an interactive tool into a Unix-style command-line utility. It reads input, processes it, writes output to stdout, and exits. This makes it composable with every other CLI tool in your workflow.

1.3 When to Use Headless Mode

Use Case	Example Command	Audience
Batch file processing	for f in *.csv; do claude -p "Validate $f"; done	Process Eng.
CI/CD code review	gh pr diff | claude -p "Review for security issues"	IT Engineers
Test generation	claude -p "Generate pytest tests for @src/calc.py"	Both
Log analysis	cat error.log | claude -p "Summarize the errors"	Both
Release notes	git log --oneline -20 | claude -p "Write release notes"	IT Engineers
Schema validation	claude -p "Check @schema.json against @data.csv"	Process Eng.
Cron job reports	claude -p "Generate weekly sensor data summary"	Process Eng.
Pre-commit hooks	claude -p "Check this diff for style violations"	IT Engineers

QUICK CHECK (1 minute)

- What flag enables headless mode? (-p or --print)
- Does headless mode open an interactive session? (No — run, return, exit)
- Name one use case relevant to your work at TI.

0:10 – 0:18	SECTION 2: Output Formats — Text, JSON & Streaming

2. Output Formats: Text, JSON & Streaming

2.1 Three Output Modes

Headless mode supports three output formats, each suited to different automation needs. Choosing the right format determines how easily you can parse, log, and integrate Claude's responses into your workflows.

Format	Flag	Best For
Text (default)	--output-format text	Human-readable output, simple scripts, saving to .md/.txt files
JSON	--output-format json	Programmatic parsing with jq, CI/CD integration, cost tracking
Streaming JSON	--output-format stream-json	Real-time progress monitoring, long-running refactoring tasks

2.2 Text Output (Default)

Text output prints Claude's response as plain text to stdout. Use this when you just need a human-readable result or want to redirect output to a file:

```
# Default text output — prints plain text
claude -p "List all Python files with more than 100 lines"

# Save to a file
claude -p "Generate a README for this project" > README.md

# Use in a conditional check
result=$(claude -p "Does config.py have hardcoded passwords? YES or NO")
if [[ "$result" == *"YES"* ]]; then
  echo "WARNING: Hardcoded passwords detected!"
  exit 1
fi
```

2.3 JSON Output

JSON output wraps the response in structured metadata — including cost, duration, session ID, and the result text. This is essential when Claude's output will be consumed by another program:

```
# Get JSON output with metadata
claude -p "How many functions are in validators.py?" \
  --output-format json

# Response structure:
# {
#   "type": "result",
#   "subtype": "success",
#   "total_cost_usd": 0.003,
#   "is_error": false,
```

```
#   "duration_ms": 1234,
#   "num_turns": 6,
#   "result": "There are 5 functions...",
#   "session_id": "abc123-def456"
# }

# Parse with jq:
resp=$(claude -p "Check for bugs" --output-format json)
text=$(echo "$resp" | jq -r '.result')
cost=$(echo "$resp" | jq -r '.total_cost_usd')
echo "Analysis: $text"
echo "Cost: \$$cost"
```

☆ **KEY POINT:** Always use --output-format json when Claude's output will be consumed by another program. The JSON envelope gives you error detection (is_error), cost tracking (total_cost_usd), and session persistence (session_id) — all essential for production automation.

2.4 Streaming JSON Output

Streaming JSON emits each message as a separate JSON object as it's generated, line by line. Use this for long-running tasks where you need real-time progress feedback:

```
# Stream messages in real-time
claude -p "Refactor all test files to use fixtures" \
  --output-format stream-json

# Each line is a separate JSON object:
# {"type":"system","subtype":"init",...}
# {"type":"assistant","message":{...}}
# {"type":"result","subtype":"success",...}

# Filter the stream with jq:
claude -p "Migrate utils.py to async" \
  --output-format stream-json | \
  jq --unbuffered -r 'select(.type=="assistant")'
```

0:18 – 0:28	SECTION 3: Piping, Chaining & Unix Integration

3. Piping, Chaining & Unix Integration

3.1 Claude Code as a Unix Citizen

One of headless mode's most powerful features is that Claude Code behaves like a standard Unix utility. It reads from stdin, writes to stdout, and returns exit codes. This means you can pipe data into it, pipe its output to other tools, and chain it with anything in your Unix toolkit.

```
# Pipe data INTO Claude Code
cat sensor_data.csv | claude -p "Find anomalous readings"

# Pipe Claude's output TO another tool
claude -p "List functions missing docstrings" | sort

# Full pipeline: find, analyze, save
git diff HEAD~5 | claude -p "Summarize these changes" > changelog.md

# Multi-step pipeline
find src/ -name "*.py" -mtime -7 | \
  xargs cat | \
  claude -p "Review recent changes for quality" > weekly-review.md
```

☆ **KEY POINT:** Piping data into Claude Code is the #1 technique for process engineers. Instead of manually opening each CSV or log file, you pipe the data directly to Claude and get analysis in seconds. This turns hours of manual inspection into a single command.

3.2 Process Engineering Pipelines

Here are practical automation patterns specifically for process engineers working with sensor data, CSV files, and manufacturing logs:

```
# 1. Validate a batch of CSV files against schema
for csv_file in data/incoming/*.csv; do
  echo "Checking: $csv_file"
  claude -p "Validate @$csv_file: columns must be
    timestamp(ISO8601), sensor_id(int), temp(float -40..150),
    pressure(float 0..1000), humidity(float 0..100).
    Report schema violations." \
    --output-format json > "reports/${csv_file##*/}.json"
done

# 2. Summarize daily sensor logs
cat /var/log/sensors/$(date +%Y-%m-%d).log | \
  claude -p "Summarize: error count, warning count,
    top 3 failing sensor IDs, recommended actions"

# 3. Generate data quality report from multiple sources
paste <(cat yield_data.csv) <(cat defect_data.csv) | \
  claude -p "Cross-reference yield and defect data.
    Flag lots where yield < 95% and defects > 2%."
```

3.3 IT Engineering Pipelines

For IT engineers, the most common patterns involve git integration, code review, and CI/CD automation:

```
# 1. Review a pull request for security issues
gh pr diff 42 | claude -p \
  "Review this diff for security vulnerabilities. \
   Focus on: SQL injection, XSS, hardcoded secrets, \
   insecure API calls. Output as JSON." \
  --output-format json > security-review.json

# 2. Generate release notes from git history
git log --oneline v1.2.0..HEAD | claude -p \
  "Write professional release notes grouped by:
   Features, Bug Fixes, Breaking Changes, Internal."

# 3. Explain a complex error for the on-call engineer
kubectl logs pod/api-server --tail=50 | claude -p \
  "Diagnose this error. What happened, root cause,
   and suggested fix. Keep it concise for on-call."
```

3.4 Multi-Turn Automated Sessions

For complex tasks that require multiple steps, use --continue to chain prompts within the same session context, or --resume with a session ID for specific conversations:

```
# Multi-step analysis with session continuity
# Step 1: Initial analysis
claude -p "Analyze src/ for performance bottlenecks" \
  --output-format json > step1.json

# Step 2: Continue the same session for follow-up
claude -p "Now focus on the database queries" --continue

# Step 3: Generate final summary
claude -p "Generate a summary of all issues found" --continue

# Or use session IDs for explicit tracking:
sid=$(jq -r '.session_id' step1.json)
claude -p "Fix the top 3 issues" --resume $sid
```

⚠ **WARNING:** Each --continue or --resume call carries the full conversation context, which uses tokens. For batch processing (looping over files), use independent -p calls instead — each file gets a clean context. Reserve --continue for genuinely multi-step reasoning tasks.

0:28 – 0:36	SECTION 4: System Prompts, Permissions & Safety

4. System Prompts, Permissions & Safety in Automation

4.1 Custom System Prompts

When running headless, you can inject custom instructions that shape Claude's behavior for every automated run. This is how you enforce team coding standards, output formats, and domain-specific rules without repeating them in every prompt:

Flag	Purpose
--append-system-prompt	Adds instructions on top of Claude's defaults. Claude keeps its built-in behavior plus your additions.
--system-prompt	Replaces the entire system prompt. Use only when you need full control over Claude's behavior.

```
# Append team standards to every automated review
gh pr diff "$PR" | claude -p \
  "Review this code for issues" \
  --append-system-prompt "You are a senior TI engineer.
    Enforce: Google-style docstrings, type hints on all
    public functions, no print() in production code,
    all constants in UPPER_SNAKE_CASE."

# Full system prompt replacement for specialized tasks
cat data.csv | claude -p "Analyze this data" \
  --system-prompt "You are a semiconductor process
    engineer. Analyze sensor data for yield anomalies.
    Output format: JSON with fields: anomaly_count,
    affected_sensors, severity, recommendation."
```

☆ **KEY POINT:** Use --append-system-prompt for 90% of automation. It keeps Claude's default safety and capabilities while adding your team-specific rules. Only use --system-prompt when you need to completely redefine Claude's persona for a specialized tool.

4.2 Permission Controls for Automation

In headless mode, there's no one to click "Approve" on permission prompts. You must pre-configure what Claude is allowed to do using --allowedTools and --disallowedTools:

```
# Allow only reading and grepping (safe for analysis)
claude -p "Find all hardcoded API keys" \
  --allowedTools "Read,Grep,Glob"

# Allow specific bash commands (controlled execution)
claude -p "Run the test suite and report results" \
  --allowedTools "Read,Bash(pytest *),Bash(python -m pytest *)"

# Block dangerous tools explicitly
claude -p "Refactor the utils module" \
```

```
  --disallowedTools "Bash(rm *),Bash(curl *),Bash(wget *)"

# For fully trusted, isolated environments (Docker/sandbox):
claude -p "Fix all linting errors" \
  --dangerously-skip-permissions
# WARNING: Only use in sandboxed containers with no secrets!
```

⚠ **WARNING:** The --dangerously-skip-permissions flag gives Claude unrestricted access to your filesystem and shell. Only use this inside isolated Docker containers, CI runners, or sandbox environments with no access to production data, secrets, or the internet.

4.3 Safety Checklist for Automated Scripts

Before deploying any headless Claude Code script in a TI environment, walk through this checklist:

✅ AUTOMATION SAFETY CHECKLIST FOR TI

- Define explicit --allowedTools for every automated script
- Never use --dangerously-skip-permissions outside sandboxed containers
- Use --output-format json to capture is_error for error handling
- Add timeout wrappers: timeout 300 claude -p "..." to prevent runaway tasks
- Log costs: parse total_cost_usd from JSON output for budget tracking
- Test scripts in a non-production repo before deploying to CI/CD
- Use --append-system-prompt for team standards (never hardcode rules in prompts)
- Store session IDs from JSON output for audit trails

4.4 Error Handling in Scripts

Production automation must handle failures gracefully. Claude Code returns non-zero exit codes on failure, and JSON output includes is_error and subtype fields:

```
#!/bin/bash
set -e

# Robust error handling pattern
if ! result=$(claude -p "Review @src/app.py for bugs" \
    --output-format json \
    --allowedTools "Read,Grep" 2>claude-error.log); then
  echo "ERROR: Claude Code failed" >&2
  cat claude-error.log >&2
  exit 0  # Don't block the pipeline
fi

# Check for errors in the JSON response
is_err=$(echo "$result" | jq -r '.is_error')
if [[ "$is_err" == "true" ]]; then
  echo "Claude returned an error" >&2
  echo "$result" | jq -r '.result' >&2
  exit 1
fi

# Extract result and cost
echo "$result" | jq -r '.result'
```

```
echo "Cost: $(echo "$result" | jq -r '.total_cost_usd')" >&2
```

0:36 – 0:43	SECTION 5: Real-World Automation Recipes for TI

5. Real-World Automation Recipes for TI

This section presents complete, copy-paste-ready automation scripts organized by role. Each recipe solves a real problem that TI engineers face daily.

5.1 Recipe: Batch CSV Validation (Process Engineers)

Validate incoming sensor data files against a known schema before they enter the data pipeline. This replaces manual spot-checking of CSV files:

```
#!/bin/bash
# validate_sensor_batch.sh
# Validates all CSV files in a directory against schema

INPUT_DIR="data/incoming"
REPORT_DIR="reports/validation"
mkdir -p "$REPORT_DIR"

SCHEMA="columns: timestamp(ISO8601), sensor_id(int),
  temp_celsius(float:-40..150), pressure_kpa(float:0..1000),
  humidity_pct(float:0..100). Flag invalid rows."

for f in "$INPUT_DIR"/*.csv; do
  echo "Validating: $(basename $f)"
  claude -p "Validate @$f against schema: $SCHEMA" \
    --output-format json \
    --allowedTools "Read" \
    --append-system-prompt "Output JSON: {valid:bool,
      total_rows:int, invalid_rows:[], summary:str}" \
    > "$REPORT_DIR/$(basename $f .csv).json"
done

echo "Validation complete. Reports in $REPORT_DIR/"
```

5.2 Recipe: Automated Code Review (IT Engineers)

Run Claude Code as a code reviewer on every pull request in your CI pipeline. This catches common issues before human reviewers spend time:

```
#!/bin/bash
# review_pr.sh - Called from GitHub Actions or GitLab CI
PR_NUMBER=$1

# Get the diff
DIFF=$(gh pr diff "$PR_NUMBER")

# Run Claude review with team standards
review=$(echo "$DIFF" | claude -p \
  "Review this pull request diff for:
   1. Bugs and logic errors
   2. Security vulnerabilities
```

```
    3. Missing error handling
    4. Style violations (Google Python style)" \
  --output-format json \
  --allowedTools "Read,Grep" \
  --append-system-prompt "You are a senior TI engineer.
    Be specific. Reference file:line for each issue.
    Rate severity: critical / major / minor.")

# Post review as PR comment
text=$(echo "$review" | jq -r '.result')
gh pr comment "$PR_NUMBER" --body "## AI Code Review\n$text"

# Fail CI on critical issues
if echo "$text" | grep -qi "critical"; then
  echo "CRITICAL issues found — blocking merge"
  exit 1
fi
```

5.3 Recipe: Daily Sensor Summary Report (Process Engineers)

Generate an executive summary from the day's sensor data, automatically emailed to the team lead at end of shift:

```
#!/bin/bash
# daily_sensor_summary.sh - runs via cron at 18:00
DATE=$(date +%Y-%m-%d)
LOG="/var/log/sensors/$DATE.log"

cat "$LOG" | claude -p \
  "Generate a daily sensor summary report including:
   - Total readings processed
   - Sensors with out-of-range values
   - Top 5 anomalies with timestamps
   - Trend analysis vs. previous 7-day average
   - Recommended actions for engineering team
   Format as Markdown with tables." \
  --append-system-prompt "You are a TI process engineer.
    Use semiconductor manufacturing terminology.
    Be data-driven: cite specific values and thresholds." \
  > "/reports/daily/$DATE-summary.md"

# Email the report
mail -s "Sensor Summary $DATE" team-lead@ti.com \
  < "/reports/daily/$DATE-summary.md"
```

5.4 Recipe: Batch Test Generation (Both Tracks)

Generate unit tests for all Python modules that currently have less than 80% coverage:

```
#!/bin/bash
# generate_missing_tests.sh

# Find modules with < 80% coverage
pytest --cov=src --cov-report=term-missing 2>/dev/null | \
```

```
  grep -E '^src/' | \
  awk -F' ' '$4 < 80 {print $1}' | \
  while read module; do
    echo "Generating tests for: $module"
    claude -p "Generate comprehensive pytest tests for @$module.
      Include: happy path, edge cases, error handling.
      Use fixtures and parametrize where appropriate.
      Follow Google testing style." \
      --allowedTools "Read,Write" \
      --output-format json > /dev/null
  done

echo "Tests generated. Run pytest to verify."
```

☆ **KEY POINT:** These recipes are templates, not finished products. The real power comes from adapting them to your specific TI workflow. Change the schema, the review criteria, the report format — the pattern stays the same: pipe data in, get analysis out, integrate into your existing tools.

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Lab 1

6. Quick Reference & Transition

MODULE 5 RECAP

✅ Headless mode (-p): transforms Claude Code into a scriptable Unix utility — run, return, exit. No interactive UI.

✅ Output formats: text (default, human-readable), json (structured with metadata), stream-json (real-time message streaming).

✅ Piping & chaining: pipe data into Claude via stdin, pipe output to other tools, chain with find/grep/git/jq in full Unix pipelines.

✅ System prompts: use --append-system-prompt for team standards, --system-prompt for complete persona replacement.

✅ Permissions: --allowedTools and --disallowedTools pre-configure what Claude can do. Only use --dangerously-skip-permissions in sandboxes.

✅ Multi-turn sessions: --continue resumes the last session, --resume <id> picks up a specific conversation for multi-step workflows.

QUICK REFERENCE CARD

Feature	Command / Flag	Notes
Headless mode	claude -p "prompt"	Non-interactive, stdout output
Text output	--output-format text	Default, human-readable
JSON output	--output-format json	Structured with cost and session_id
Stream JSON	--output-format stream-json	Real-time message streaming
Pipe stdin	cat file | claude -p "..."	Send data to Claude via stdin
Save output	claude -p "..." > file.md	Redirect stdout to file
Append system	--append-system-prompt "..."	Add to default system prompt
Replace system	--system-prompt "..."	Replace entire system prompt
Allow tools	--allowedTools "Read,Grep"	Whitelist specific tools
Block tools	--disallowedTools "Bash(rm *)"	Blacklist dangerous tools
Skip permissions	--dangerously-skip-permissions	ONLY in sandboxed environments
Continue session	--continue	Resume most recent session
Resume by ID	--resume <session-id>	Resume specific conversation
Verbose mode	--verbose	Debug output for troubleshooting
Timeout wrapper	timeout 300 claude -p "..."	Prevent runaway executions

TRANSITION TO MODULE 6: LAB 1

You now know how to script Claude Code for automation. In Module 6 (Lab 1), you'll put these skills to work hands-on: initialize a new project with /init, scaffold a Python project structure with CLAUDE.md,

generate core modules, and write your first automated tests — all using the commands and patterns you've learned in Modules 1–5.

Track A (IT engineers): You'll scaffold a project with advanced pytest fixtures, parametrized tests, and CI-ready configuration.

Track B (Process engineers): You'll scaffold a data processing project with basic test patterns, CSV validation, and automated reporting.